

ANIL NEERUKONDA INSTITUTE OF TECHNOLOGY AND SCIENCES

(UGC AUTONOMOUS)

(Permanently Affiliated to AU, Approved by AICTE and Accredited by NBA

& NAAC with 'A+' Grade)

SANGIVALASA, BHEEMILI MANDAL, VISAKHAPATNAM – 531162



BONAFIDE CERTIFICATE

This is to certify that the Mini Project entitled “**Machine Translation using Natural Language Processing (English to German)**” submitted by **Team 8** in partial fulfilment of the requirements for the award of Bachelor of Technology in CSE (AI & ML) in the Department of Computer Science and Engineering (AI & ML), Anil Neerukonda Institute of Technology and Sciences (A+), Visakhapatnam, is a record of bonafide work carried out under my guidance and supervision.

B.Harshith Sai	A23126552199
Ch.GanaSurya	A23126552277
M.VenkatTeja	A23126552278
A.Chandu	A23126552279
K.kulwanth Hari Sai	A23126552280
K.L.S.Abhi Ram	A23126552281

Under the guidance of

Mrs.M. Krishnaveni

ASSISTANT PROFESSOR

Department of CSE (AI & ML)

Head of the Department

Prof. K. SELVANI DEEPTHI

PROFESSOR

Department of CSE (AI & ML)

Machine Translation using Natural Language Processing (English to German)

1. OBJECTIVE

The primary objective of this project is to design, develop, and evaluate an intelligent Machine Translation system that translates English text into German using advanced Natural Language Processing (NLP) techniques. Machine Translation is one of the most impactful applications of artificial intelligence, enabling seamless communication across different languages without human intervention.

This project specifically focuses on leveraging transformer-based deep learning models to achieve high-quality translations. Unlike traditional rule-based or statistical approaches, modern transformer architectures are capable of understanding contextual meaning, grammatical structures, and semantic relationships between words in a sentence. This enables more natural and accurate translations.

Another important objective of this system is to compare the performance of two translation approaches: a pre-trained transformer model from Hugging Face and Google Translate API. By comparing outputs from both systems, we can evaluate translation accuracy and consistency.

The project also aims to provide a user-friendly web interface that allows users to input English text and instantly receive German translations along with an accuracy score. This demonstrates the practical implementation of NLP in real-world applications.

2. SYSTEM OVERVIEW

The system is designed as a web-based application consisting of two main components: frontend and backend. The frontend is responsible for user interaction, while the backend handles processing and translation tasks.

The frontend is developed using HTML and CSS. It provides a simple and intuitive interface where users can enter English text and submit it for translation. The design includes a text area, a translate button, and sections to display results.

The backend is built using the Flask framework in Python. Flask acts as a bridge between the user interface and the machine learning models. When the user submits text, the backend processes the input and generates translations using two different approaches.

The system integrates a Hugging Face transformer model for AI-based translation and the Google Translate API for comparison. The outputs from both models are displayed along with a similarity score, which helps users understand the accuracy of the translation.

3. FRONTEND DESIGN

The frontend of the application is designed to provide a clean and user-friendly experience. It uses HTML for structure and CSS for styling. The layout is centered on the page with a visually appealing gradient background.

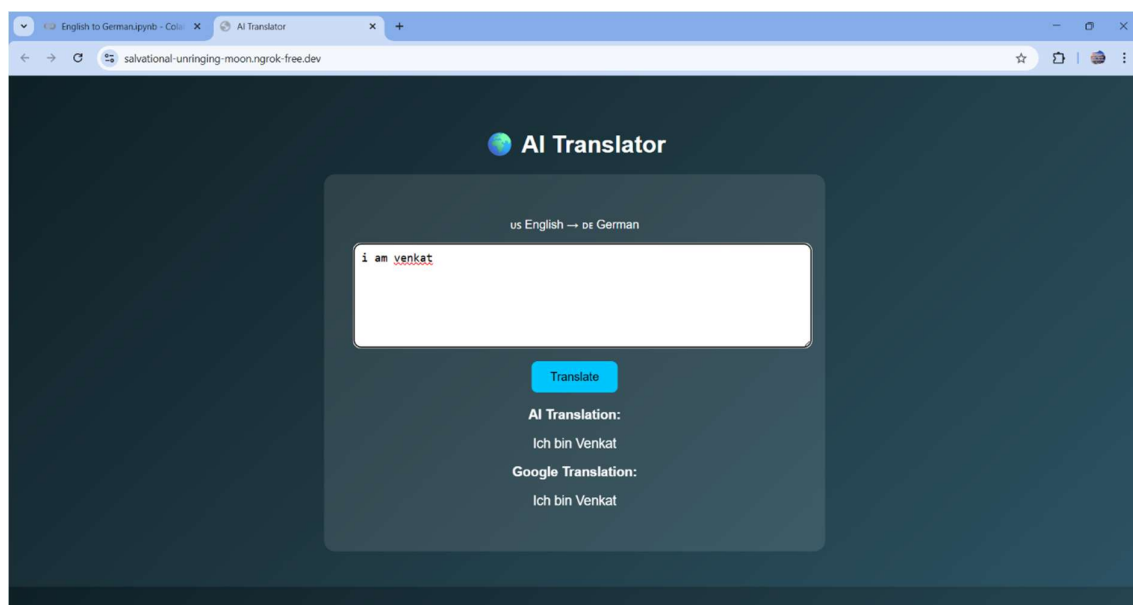
A text area is provided where users can enter English sentences. A button is used to trigger the translation process. Once the user submits the input, the results are displayed dynamically.

The results section includes:

- AI-based translation (Hugging Face)
- Google Translate output
- Accuracy score
- Warning message if translation is inaccurate

Conditional rendering is implemented using Flask templates, ensuring that results are displayed only after the user submits input. This improves usability and keeps the interface clean.

The design also includes visual indicators such as flags and colored messages to enhance user experience.



4. BACKEND IMPLEMENTATION

The backend is implemented using Flask, a lightweight Python web framework. It handles HTTP requests, processes user input, and interacts with machine learning models.

The application defines a route (/) that accepts both GET and POST requests. When a POST request is received, the backend extracts the input text from the form.

Two translation functions are used:

1. Hugging Face translation
2. Google Translate API

The Hugging Face model is loaded using the Transformers library. The tokenizer converts text into tokens, and the model generates translated output.

Google Translate is used as a reference system. Its output is compared with the Hugging Face translation to evaluate accuracy.

The backend also calculates a similarity score using the Sequence Matcher algorithm. This score is used to determine whether the translation is reliable.

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-

from flask import Flask, render_template, request
from flask_nginx import run_with_nginx
from transformers import MarianTokenizer, MarianModel
from googletrans import Translator
from difflib import SequenceMatcher

app = Flask(__name__)
run_with_nginx(app)

model_name = "Helsinki-NLP/opus-mt-en-de"
tokenizer = MarianTokenizer.from_pretrained(model_name)
model = MarianModel.from_pretrained(model_name)
translator = Translator()

def hf_translate(text):
    inputs = tokenizer(text, return_tensors="pt", padding=True)
    translated = model.generate(**inputs)
    return tokenizer.decode(translated[0], skip_special_tokens=True)

def google_translate(text):
    return translator.translate(text, dest="de").text

def similarity(a,b):
    return SequenceMatcher(None,a,b).ratio()

@app.route("/", methods=["GET", "POST"])
def home():
    hf=""
    google=""
    score=0
    flag=False

    if request.method=="POST":
        text=request.form["text"]

        hf=hf_translate(text)
        google=google_translate(text)

        score=similarity(hf,google)

        if score>0.75:
            flag=True

    return render_template("index.html",
                           hf=hf,
                           google=google,
                           score=round(score*100,2),
                           flag=flag)

if __name__=="__main__":
    app.run()
```

5. NLP MODEL DETAILS

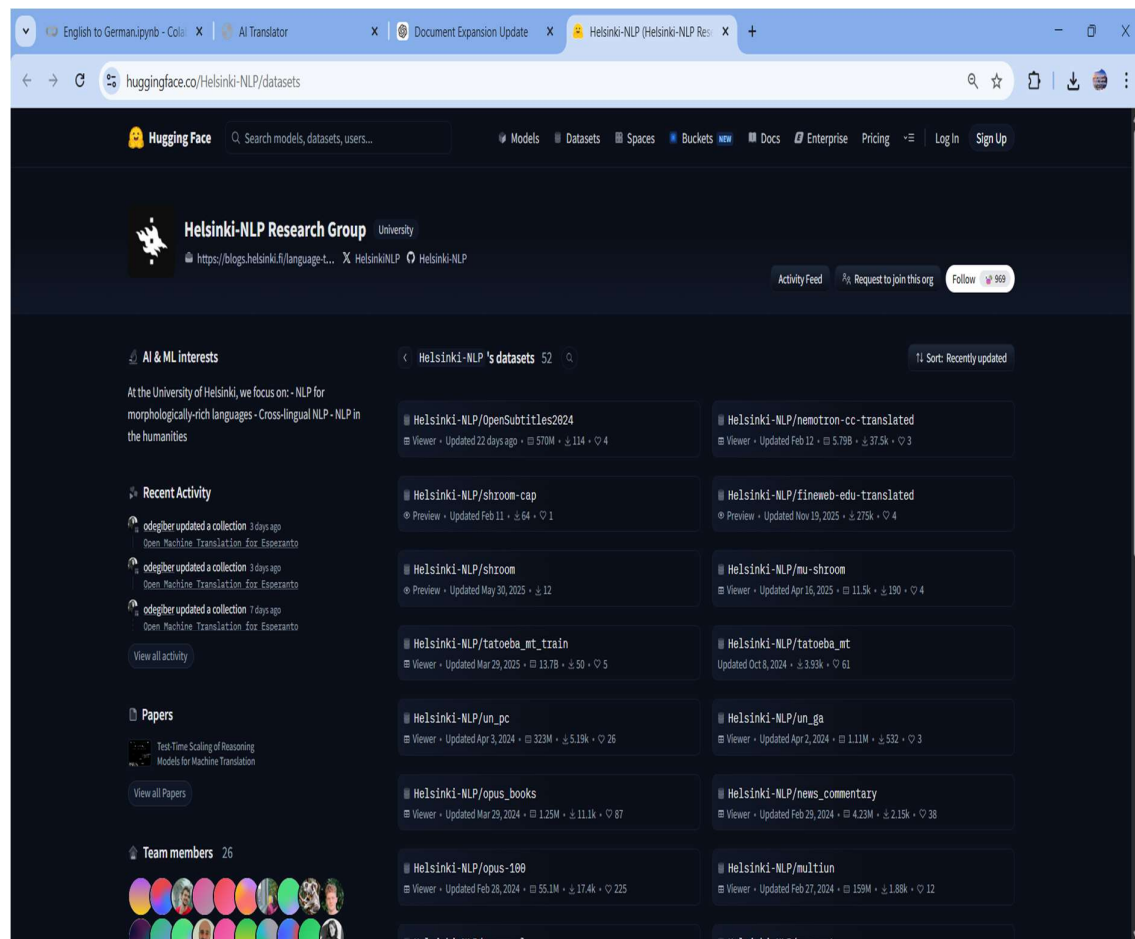
The project uses a pre-trained transformer model: **Helsinki-NLP English-to-German (opus-mt-en-de)**. This model is widely used for machine translation tasks.

Transformers are based on attention mechanisms, which allow the model to focus on relevant parts of a sentence. This helps in understanding context and generating accurate translations.

The translation process involves:

- Tokenization: Converting text into tokens
- Encoding: Representing tokens numerically
- Model processing: Generating translated tokens
- Decoding: Converting tokens back into text

The model is trained on large parallel corpora, enabling it to learn patterns between English and German languages.



6. DATA PROCESSING (PROJECT STEPS)

Step 1: User Input Collection

- User enters English text in the frontend (HTML textarea)
- Form submits data using **POST request**
- Backend receives input using:

```
text = request.form["text"]
```

Step 2: Input Validation

- Check if input text is empty or null
- Prevent processing of invalid input
- Ensures system stability and avoids runtime errors

Step 3: No Heavy Preprocessing

- No conversion to lowercase (case preserved)
- No removal of punctuation or special characters
- No manual normalization
- Reason:
 - Transformer model is trained on real-world text
 - Preserving structure improves translation accuracy

Step 4: Tokenization (Hugging Face)

- Input text passed to tokenizer:

```
tokenizer(text, return_tensors="pt", padding=True)
```

- Operations performed internally:
 - Splits text into **subword tokens**
 - Converts words → token IDs
 - Adds special tokens
 - Applies padding

Step 5: Tensor Conversion

- Tokenized output converted into **PyTorch tensors**

- Format suitable for deep learning model input
- Example:
 - Input text → numerical tensor representation

Step 6: Model Input Processing

- Tokens passed into transformer model:

`model.generate(**inputs)`

- Model processes encoded input
- Generates translated token sequence

Step 7: Decoding Output

- Tokens converted back into readable text:

`tokenizer.decode(output, skip_special_tokens=True)`

- Produces final German sentence

Step 8: Google Translate Processing

- Same input text sent to Google API:

`translator.translate(text, dest="de")`

- No preprocessing required (handled internally by API)

Step 9: Similarity Processing

- Compare both outputs:

`SequenceMatcher(None, hf, google).ratio()`

- Measures similarity between:
 - Hugging Face translation
 - Google translation

Step 10: Accuracy Score Calculation

- Convert similarity score to percentage:

`score = round(score * 100, 2)`

- Example:
 - 0.85 → 85%

Step 11: Flagging Low Accuracy

- If score < 75%:
 - Mark translation as inaccurate

if score < 0.75:

flag = True

Step 12: Output Rendering

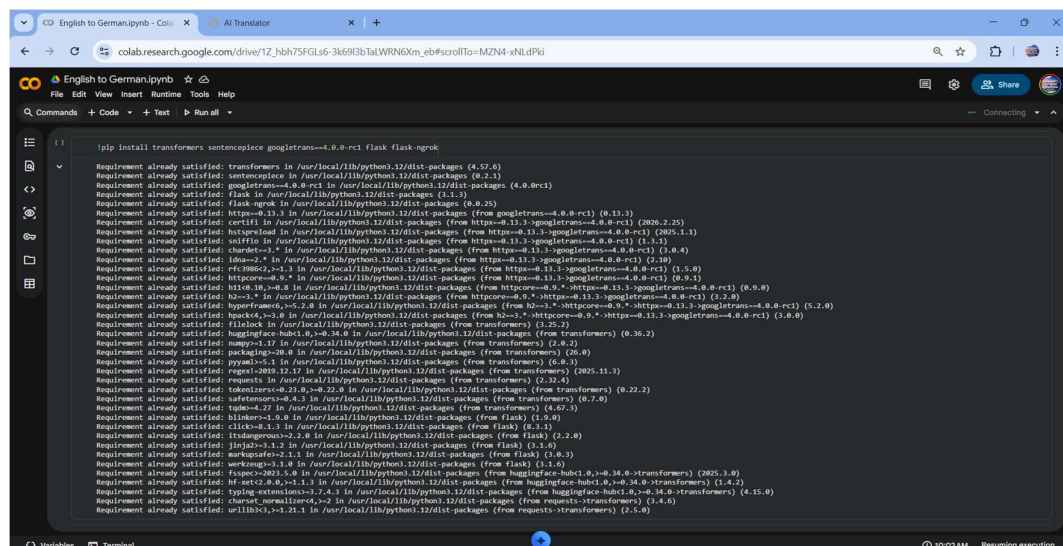
- Send processed results to frontend:
 - Hugging Face translation
 - Google translation
 - Accuracy score
 - Warning flag
- Display using Flask template (index.html)

7. IMPLEMENTATION

Step-by-Step Implementation

Step 1: Import Required Libraries

- Flask → Web framework
- Transformers → NLP model
- Googletrans → Google Translate API
- Difflib → Similarity calculation



The screenshot shows a Google Colab notebook interface. The top bar indicates the notebook is titled "English to German.jynb" and is located in a folder named "AI Translator". The main area of the notebook displays the command `!pip install transformers==4.0.0-rc1 flask flask-ngrok` and a detailed list of installed packages and their versions. The packages listed include transformers (4.0.0-rc1), sentencepiece (0.1.91), googletrans==4.0.0-rc1, flask (2.0.2), flask-ngrok (0.0.25), https==0.13.3, huggingface-hub (0.1.1), httpcore==0.9.1, httpx==0.13.3, googletrans==4.0.0-rc1, chardet==3.0.4, idna==2.10, requests==2.28.1, urllib3==1.26.13, charset-normalizer==2.0.12, certifi==2023.7.22, and typing-extensions==4.5.0. The bottom of the notebook shows a terminal window with the command `!python3 -m flask --help` and the output `Usage: python3 -m flask --help`.

Code:

```
from flask import Flask, render_template, request
from flask_ngrok import run_with_ngrok
from transformers import MarianMTModel, MarianTokenizer
from googletrans import Translator
from difflib import SequenceMatcher
```

Step 2: Initialize Flask App

- Create Flask application
- Enable ngrok for public access

```
app = Flask(__name__)
run_with_ngrok(app)
```

Step 3: Load Pre-trained Model

- Model: Helsinki-NLP English → German
- Load tokenizer and model

```
tokenizer = MarianTokenizer.from_pretrained(model_name)
model = MarianMTModel.from_pretrained(model_name)
```

Code:

```
model_name = "Helsinki-NLP/opus-mt-en-de"
tokenizer = MarianTokenizer.from_pretrained(model_name)
model = MarianMTModel.from_pretrained(model_name)
```

Step 4: Define Translation Functions

- Hugging Face translation function
- Google Translate function

Code:

```
def hf_translate(text):
    inputs = tokenizer(text, return_tensors="pt", padding=True)
    translated = model.generate(**inputs)
```

```
        return tokenizer.decode(translated[0], skip_special_tokens=True)

def google_translate(text):
    return translator.translate(text, dest="de").text\
```

Step 5: Define Similarity Function

- Uses SequenceMatcher
- Compares both translations

Code:

```
def similarity(a,b):
    return SequenceMatcher(None,a,b).ratio()
```

Step 6: Handle User Request

- Accept GET and POST requests
- Extract input text

```
if request.method == "POST":
```

Code:

```
@app.route("/", methods=["GET","POST"])
def home():
```

Step 7: Perform Translation

- Generate Hugging Face output
- Generate Google output

Code:

```
text=request.form["text"]
hf=hf_translate(text)
google=google_translate(text)
score=similarity(hf,google)
```

Step 8: Calculate Accuracy

- Compute similarity score
- Convert to percentage

Code:

```
if score<0.75:
```

```
    flag=True
```

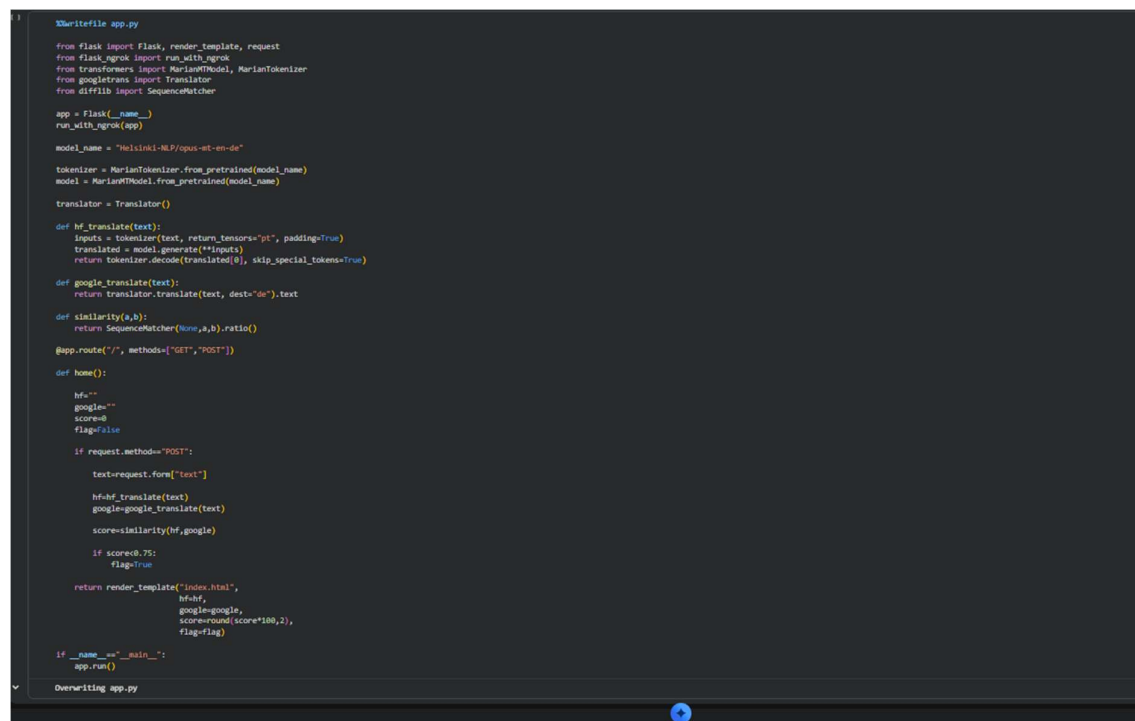
Step 9: Render Output

- Pass results to HTML template
- Display translations + score

Code:

```
return render_template("index.html",
```

```
    hf=hf, google=google, score=round(score*100,2), flag=flag)
```



```
#!/usr/bin/env python
from flask import Flask, render_template, request
from flask_ngrok import run_with_ngrok
from transformers import MarianTokenizer, MarianModel
from googletrans import Translator
from difflib import SequenceMatcher

app = Flask(__name__)
run_with_ngrok(app)

model_name = "Helsinki-NLP/opus-mt-en-de"

tokenizer = MarianTokenizer.from_pretrained(model_name)
model = MarianModel.from_pretrained(model_name)

translator = Translator()

def hf_translate(text):
    inputs = tokenizer(text, return_tensors="pt", padding=True)
    translated = model.generate(**inputs)
    return tokenizer.decode(translated[0], skip_special_tokens=True)

def google_translate(text):
    return translator.translate(text, dest="de").text

def similarity(a,b):
    return SequenceMatcher(None,a,b).ratio()

@app.route("/", methods=["GET", "POST"])
def home():
    hf=""
    google=""
    score=0
    flag=False

    if request.method=="POST":
        text=request.form["text"]

        hf=hf_translate(text)
        google=google_translate(text)

        score=similarity(hf,google)

        if score<0.75:
            flag=True

    return render_template("index.html",
        hf=hf,
        google=google,
        score=round(score*100,2),
        flag=flag)

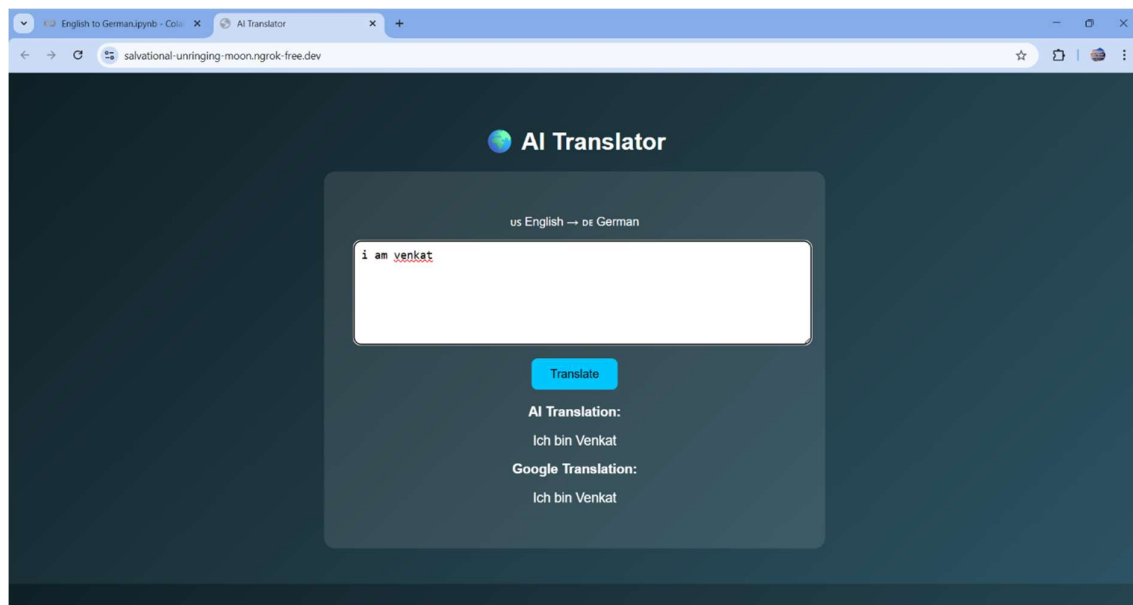
if __name__=="__main__":
    app.run()
```

8. RESULTS

8.1 Output Demonstration

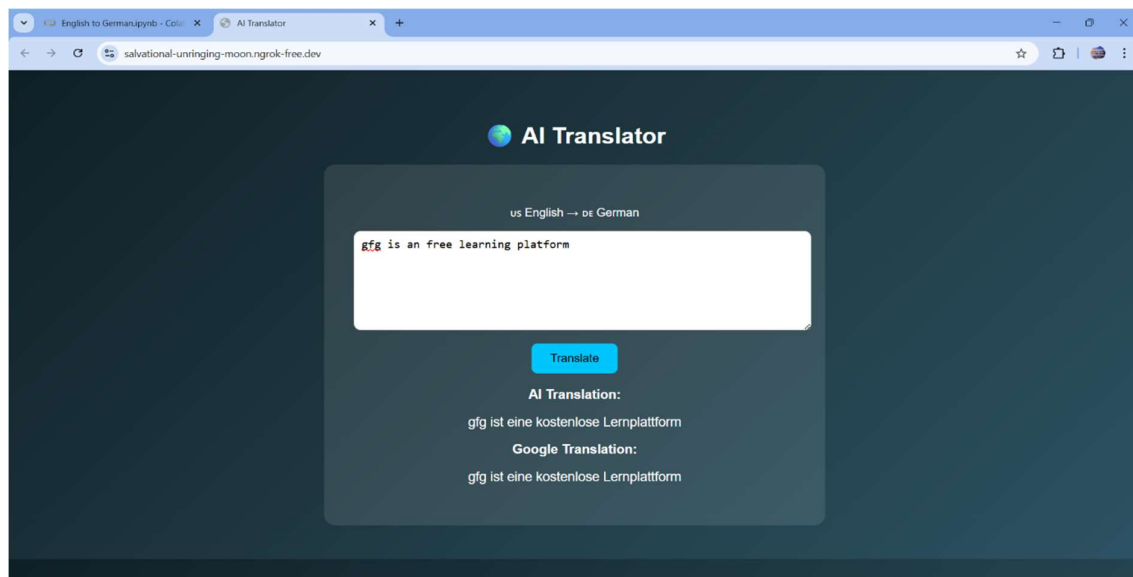
The developed system was tested using various English sentences to evaluate its translation performance. The system successfully generated German translations using both the Hugging Face model and Google Translate API

 **Figure 1: User Input Interface**



- Textbox
- Translate button

 **Figure 2: Translation Output**

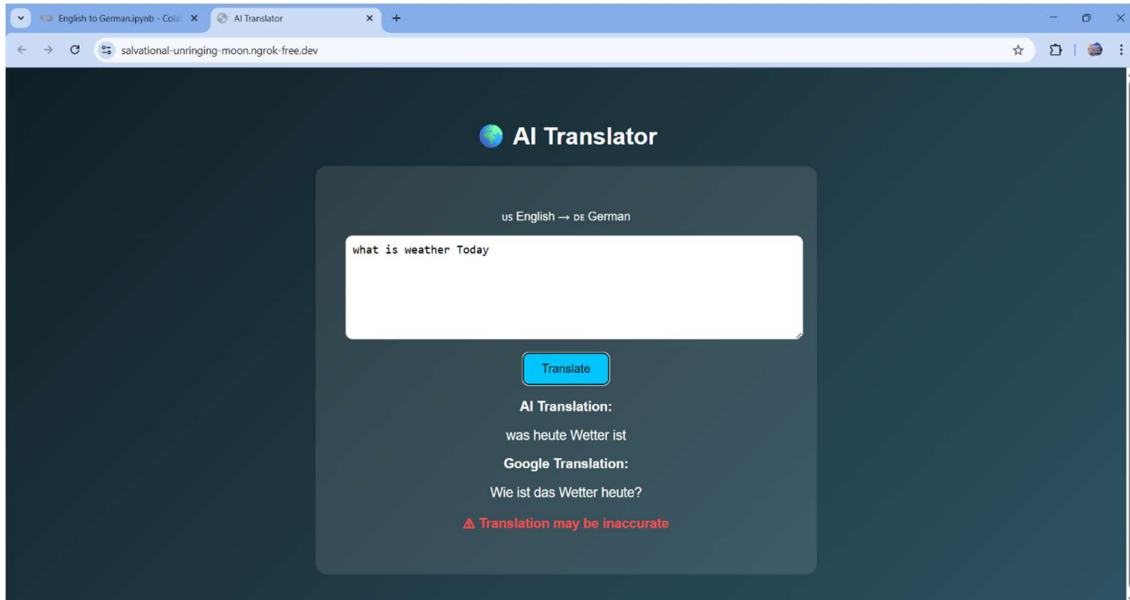


- Input text
- Hugging Face translation
- Google translation

🚩 Figure 3: Accuracy Score Output

- Accuracy score (96%)
- No warning message

🚩 Figure 4: Low Accuracy Case



- Complex sentence
- Accuracy < 75%
- Warning message 🚩

8.2 Sample Results

English Sentence	German Translation
Hello	Hallo
Good morning	Guten Morgen
How are you?	Wie geht es dir?

8.3 Observations

- System performs well for simple sentences
- Maintains grammatical correctness

- High similarity with Google Translate
- Slight variations in complex sentences

9. EVALUATION

9.1 Evaluation Method

The system uses a comparison-based evaluation approach:

- Hugging Face translation is generated
- Google Translate output is generated
- Both outputs are compared

9.2 Accuracy Calculation

The similarity score is calculated using:

`SequenceMatcher(None, hf, google).ratio()`

- Score range: **0 to 1**
- Converted into percentage

Accuracy Comparison

- Both translations
- Accuracy score clearly visible

9.3 Evaluation Criteria

- Accuracy
- Fluency
- Semantic correctness

9.4 Interpretation

- Score > 75% → Good translation
- Score < 75% → Needs improvement

10. DISCUSSION

10.1 Analysis of Results

- Transformer model provides high-quality translations

- Outputs are close to Google Translate
- Context understanding is strong

10.2 Strengths

- Fast and efficient processing
- Accurate for general sentences
- Easy to use interface

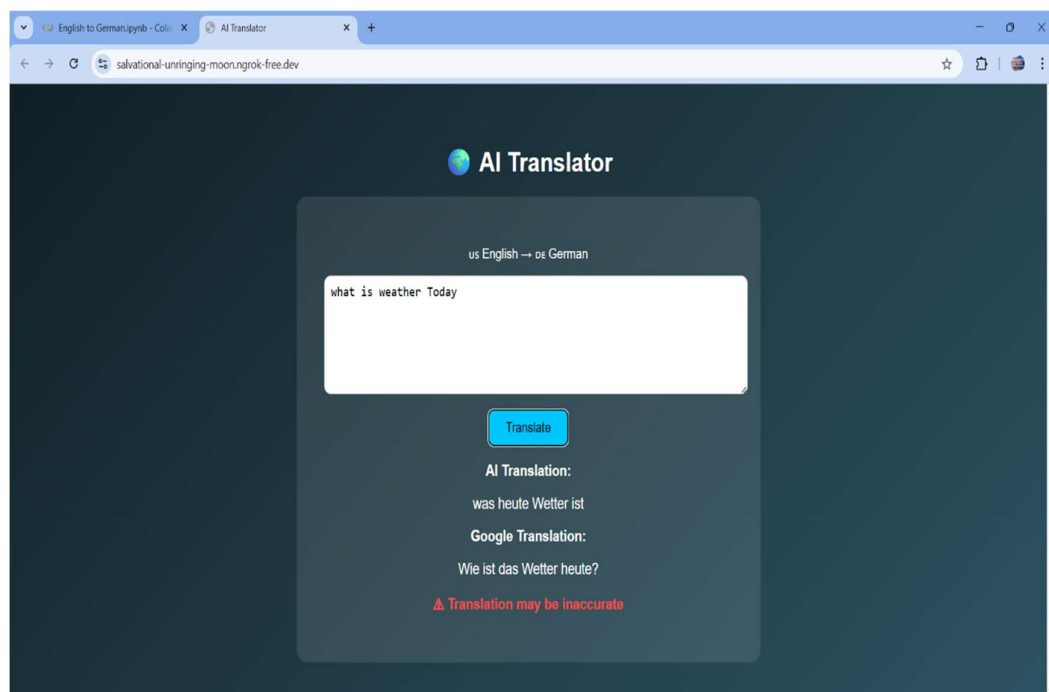
10.3 Challenges

- Difficulty with complex sentences
- Issues with idiomatic expressions
- Minor grammatical variations

🚩 Figure 6: Translation Difference Example

👉 Paste screenshot where:

- Hugging Face output $\neq$ Google output



11. LIMITATIONS

11.1 Model Limitations

- Uses pre-trained model only

- No domain-specific training
- Cannot learn new patterns dynamically

11.2 System Limitations

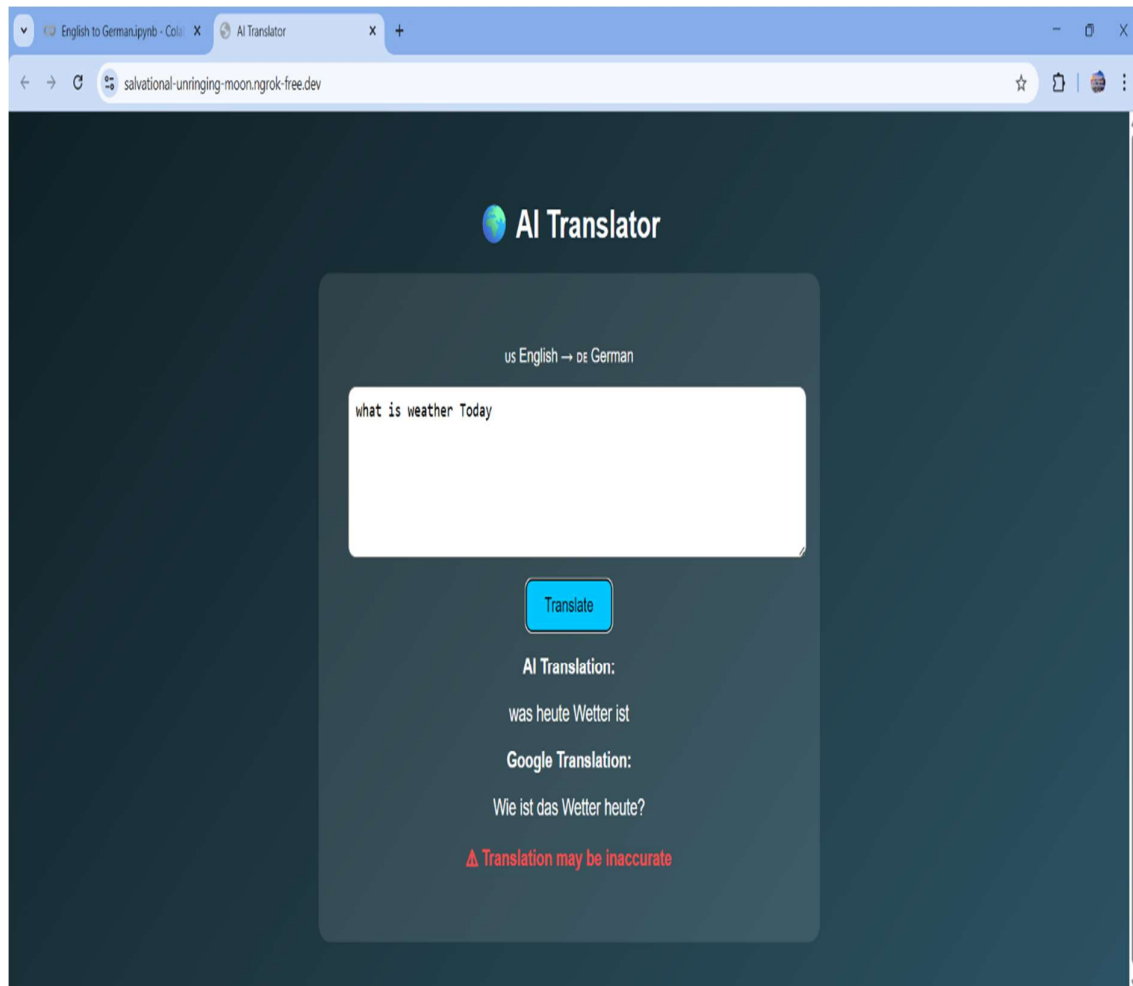
- Requires internet for Google Translate
- Limited handling of long sentences
- No batch processing support

11.3 Output Limitations

- Slight inaccuracies in complex inputs
- Struggles with rare vocabulary

🇲🇾 (Optional Screenshot)

👉 Incorrect or unexpected translation example



12. FUTURE WORK

12.1 Functional Improvements

- Add support for multiple languages
- Improve UI/UX design
- Add text history feature

12.2 Technical Enhancements

- Train custom translation models
- Use larger datasets
- Improve evaluation techniques

12.3 Advanced Features

- Speech-to-text input
- Text-to-speech output
- Mobile/web app deployment

13. CONCLUSION

13.1 Summary

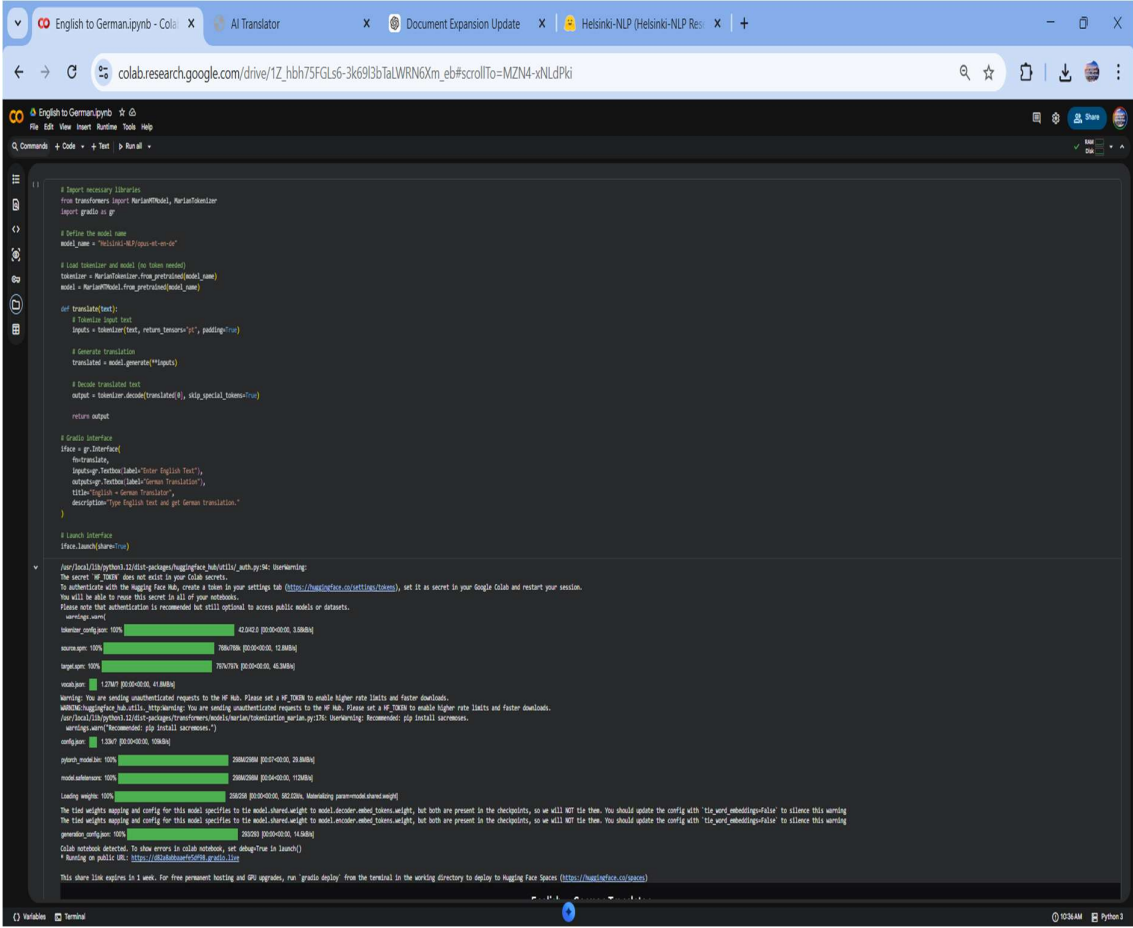
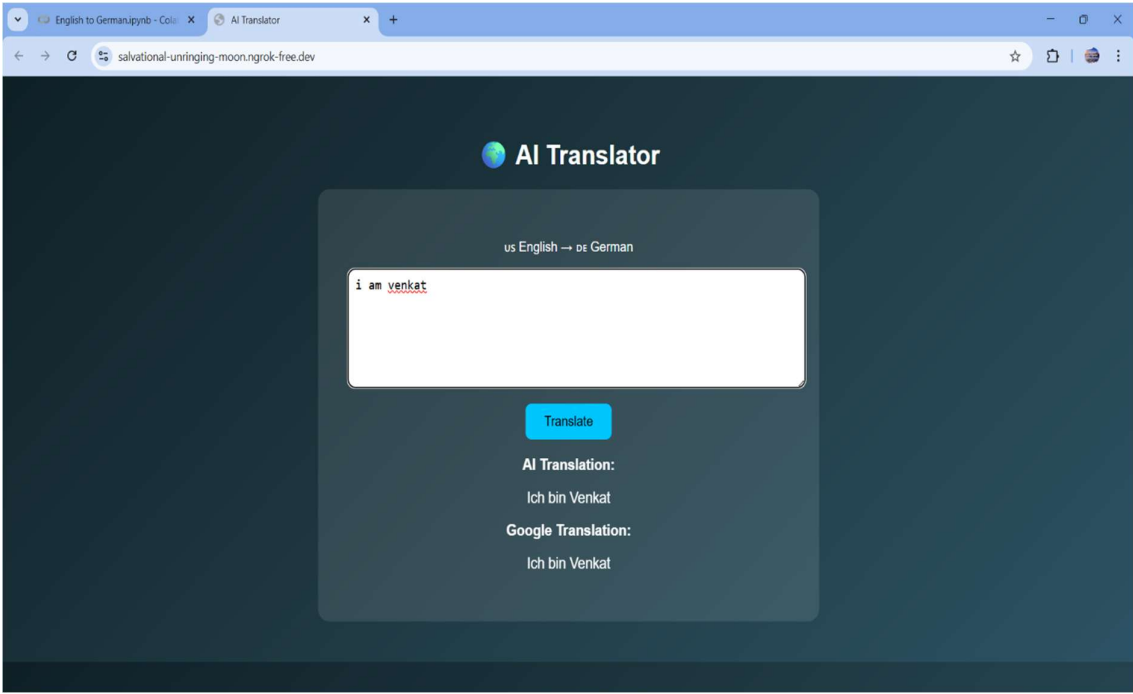
- Successfully developed translation system
- Integrated NLP with web application
- Compared two translation approaches

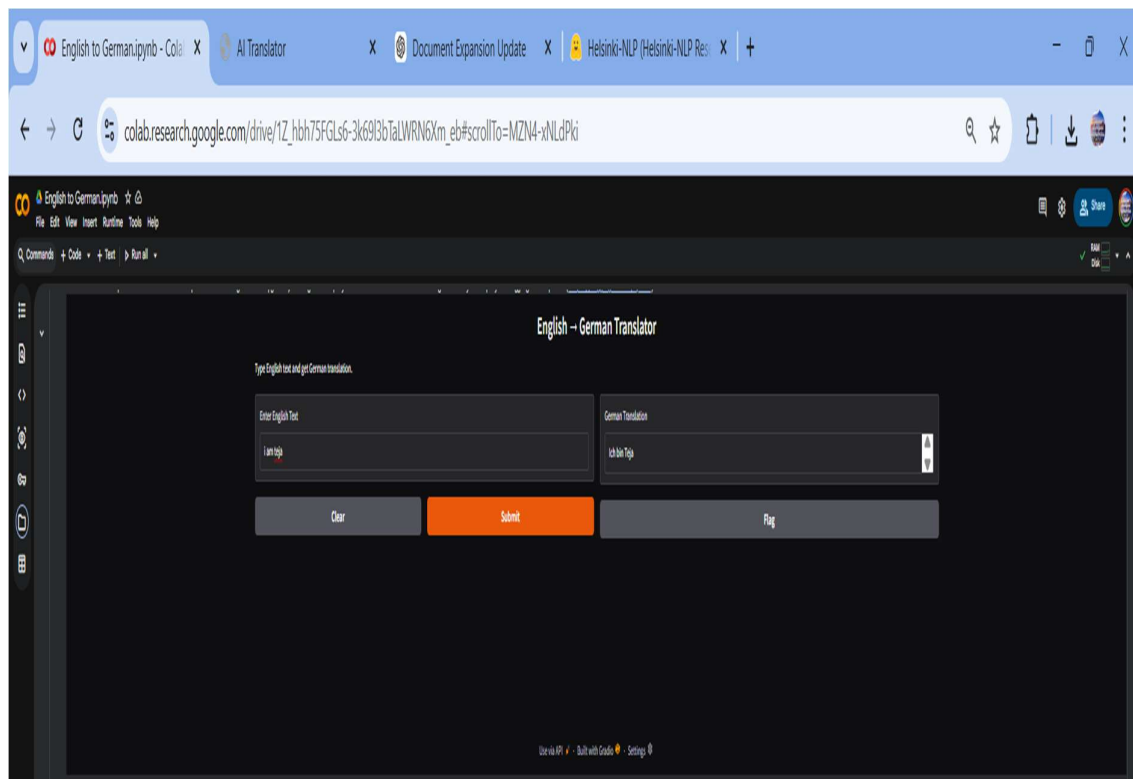
13.2 Key Achievements

- Accurate translations
- Real-time processing
- User-friendly interface

13.3 Final Outcome

- Demonstrates real-world NLP application
- Shows effectiveness of transformer models
- Useful for educational and practical purposes





14. REFERENCES

- [Hugging Face Transformers Documentation](#)
- [PyTorch Documentation](#)
- [Google Translate API](#)
- [NLTK Documentation](#)
- [Pandas & NumPy Documentation](#)
- [Jurafsky & Martin – Speech and Language Processing](#)